Компьютерное моделирование

Шпаргалка по лабораторным: условие, идея, формулы и код с комментариями

1. Лаб. 1 — События и их обработка `lab1.cpp`
2. Лаб. 2 — Система с отказами `lab2_3.cpp`
3. Лаб. 3 — Система с очередью, время ожидания `lab3_1.cpp`
4. Лаб. 4 — Суперпозиция потоков `lab4.cpp`
5. Лаб. 5 — Двухфазная система `lab5.cpp`

Общая схема событийного моделирования

1) храним моменты ближайших событий (приход, уход и т. д.); 2) выбираем самое раннее; 3) переводим часы на этот момент и обрабатываем событие; 4) планируем следующие события; 5) повторяем, пока не набрали нужное число заданий.

Равномерное U(a, b): среднее (a + b) / 2. Интенсивность = 1 / средний интервал. Загрузка ρ = работа на одно задание / средний интервал между заданиями.

Лаб. 1 — События и их обработка

`lab1.cpp`

Задача

Сгенерировать 100 случайных событий (тип от 1 до 4, время от 0 до 100), отсортировать их по времени и по очереди вызвать обработчик, соответствующий типу события.

Как работает

- Это основа событийного моделирования: события обрабатываются строго в порядке наступления.
- Сортировка вставками: каждый элемент сдвигается влево, пока слева не окажется более раннее событие.
- **handleEvent** по типу выбирает одну из четырёх функций-обработчиков.

Код

```cpp
#include <iomanip>     // setprecision — сколько знаков после запятой печатать
#include <iostream>    // cout — вывод в консоль
#include <random>      // генераторы случайных чисел

using namespace std;

struct Event {         // событие = тип + момент времени
    int type;          // тип события (1..4)
    double time;       // когда событие происходит
};

const int event_count = 100;      // сколько событий генерируем
const int type_min = 1;           // типы событий от 1 до 4
const int type_max = 4;
const double time_min = 0.0;      // время события от 0 до 100
const double time_max = 100.0;

// обработчики: на каждый тип события своя функция
void handleEvent1(double time) {
    cout << "Произошло событие 1, время = " << time << "\n";
}

void handleEvent2(double time) {
    cout << "Произошло событие 2, время = " << time << "\n";
}

void handleEvent3(double time) {
    cout << "Произошло событие 3, время = " << time << "\n";
}

void handleEvent4(double time) {
    cout << "Произошло событие 4, время = " << time << "\n";
}

// по типу события вызываем нужный обработчик
```

```cpp
void handleEvent(int type, double time) {
    if (type == 1) {
        handleEvent1(time);
    } else if (type == 2) {
        handleEvent2(time);
    } else if (type == 3) {
        handleEvent3(time);
    } else if (type == 4) {
        handleEvent4(time);
    }
}

// заполняем массив случайными событиями
void generateEvents(Event events[], int count) {
    random_device rd;                                       // источник случайного зерна
    mt19937 gen(rd());                                      // генератор (вихрь Мерсенна)
    uniform_int_distribution<int> typeDist(type_min, type_max);       // целое от 1 до 4
    uniform_real_distribution<double> timeDist(time_min, time_max);   // дробное от 0 до 100

    for (int i = 0; i < count; ++i) {
        events[i].type = typeDist(gen);      // случайный тип
        events[i].time = timeDist(gen);      // случайное время
    }
}

// сортировка вставками по времени: события должны идти в порядке наступления
void sortByTime(Event events[], int count) {
    for (int i = 1; i < count; ++i) {
        Event key = events[i];               // элемент, который вставляем на место
        int j = i - 1;
        while (j >= 0 && events[j].time > key.time) {
            events[j + 1] = events[j];       // сдвигаем более поздние события вправо
            --j;
        }
        events[j + 1] = key;                 // ставим элемент на своё место
    }
}

void printEvents(const Event events[], int count) {
    for (int i = 0; i < count; ++i) {
        cout << "  тип = " << events[i].type
             << ", время = " << events[i].time << "\n";
    }
    cout << "\n";
}

int main() {
    cout << fixed << setprecision(3);        // печатать 3 знака после запятой

    Event events[event_count];               // массив из 100 событий

    generateEvents(events, event_count);     // 1) сгенерировали
    cout << "исходное множество:\n";
    printEvents(events, event_count);

    sortByTime(events, event_count);         // 2) отсортировали по времени
    cout << "отсортированное множество:\n";
    printEvents(events, event_count);

    cout << "Функции по событиям:\n";        // 3) обработали по порядку
    for (int i = 0; i < event_count; ++i) {
        handleEvent(events[i].type, events[i].time);
    }

    return 0;
}
```

Лаб. 2 — Система с отказами

`lab2_3.cpp`

Задача

Один сервер, очереди нет. Задания приходят через τ ~ U(1, 10), обработка занимает σ ~ U(1, 5). Если задание пришло, когда сервер занят, — отказ. Считаем число отказов.

Как работает

- Два события: **IN** (приход) и **OUT** (конец обработки). На каждом шаге берём то, что наступит раньше, и переводим часы **currentTime** на этот момент.
- IN: планируем следующий приход; если сервер свободен — занимаем его, иначе отказ.
- OUT: освобождаем сервер; следующая обработка начнётся с ближайшим приходом.

Код

```cpp
#include <algorithm>
#include <iomanip>
#include <iostream>
#include <random>

using namespace std;

const int in = 1;              // код события «пришло задание»
const int out = 2;             // код события «сервер закончил обработку»

const int maxTasks = 100;      // сколько заданий моделируем

const double arrivalFrom = 1.0;    // интервал между приходами ~ U(1, 10)
const double arrivalTo = 10.0;

const double serviceFrom = 1.0;    // время обработки ~ U(1, 5)
const double serviceTo = 5.0;

// название события для вывода
const char* getEventName(int event) {
    if (event == in) {
        return "IN";
    }
    return "OUT";
}

int main() {

    cout << fixed << setprecision(2);

    random_device randomSource;
    mt19937 randomGenerator(randomSource());

    uniform_real_distribution<double> arrivalGenerator(    // генератор интервалов прихода
        arrivalFrom, arrivalTo
    );

    uniform_real_distribution<double> serviceGenerator(    // генератор времени обработки
        serviceFrom, serviceTo
    );

    bool busy = false;             // занят ли сервер
    int rejectedTasks = 0;         // сколько заданий получили отказ (очереди нет!)
    int currentTask = 1;           // номер очередного задания
    double currentTime = 0.0;      // модельное время

    double incomingTime = arrivalGenerator(randomGenerator);                  // ближайший приход
    double outgoingTime = incomingTime + serviceGenerator(randomGenerator);   // ближайший уход

    while (currentTask <= maxTasks) {

        int eventType;

        // какое событие наступит раньше — то и обрабатываем
        if (incomingTime <= outgoingTime) {
            eventType = in;
            currentTime = incomingTime;
        }
        else {
            eventType = out;
            currentTime = outgoingTime;
        }
```

```
63
64          if (eventType == in) {
65
66              incomingTime += arrivalGenerator(randomGenerator);   // планируем следующий приход
67
68              if (!busy) {
69                  busy = true;                                     // сервер свободен — берём в работу
70
71                  cout << "время = " << currentTime
72                       << ", " << getEventName(in)
73                       << ", id = " << currentTask << "\n";
74              }
75              else {
76                  rejectedTasks++;                                 // сервер занят — отказ
77
78                  cout << "время = " << currentTime
79                       << ", отказано"
80                       << ", id = " << currentTask << "\n";
81              }
82
83              currentTask++;
84          }
85          else {
86
87              busy = false;                                        // сервер освободился
88
89              cout << "время = " << currentTime
90                   << ", " << getEventName(out) << "\n";
91
92              // следующее задание начнёт обрабатываться, когда придёт, — от него и считаем уход
93              outgoingTime = incomingTime + serviceGenerator(randomGenerator);
94          }
95      }
96
97      cout << "\n событий IN отказано: " << rejectedTasks << "\n";
98
99      return 0;
100 }
```

Лаб. 3 — Система с очередью, время ожидания

`lab3_1.cpp`

Задача

Один сервер с очередью. τ ~ U(1, 10), σ ~ U(1, 8), 10 000 заданий. Из очереди сервер берёт **случайное** задание. Найти распределение времени ожидания W(x), мат. ожидание, дисперсию, СКО, долю заданий без ожидания; график F(x) — в Excel.

Как работает

- Как в лаб. 2, только занятый сервер не отказывает: задание встаёт в очередь **queue**.
- Ожидание = момент взятия в работу − момент прихода. Если сервер был свободен — ожидание 0.
- 1e100 у outgoingTime означает «сервер свободен, ухода не будет».
- Гистограмма waiting[0..9] — столбцы шириной 1, накопленная доля даёт функцию распределения F(x).

Формулы

Mτ = (a + b) / 2, Mσ = $(a_\sigma + b_\sigma) / 2$ — средние равномерного распределения

λ = 1 / Mτ — интенсивность прихода, μ = 1 / Mσ — интенсивность обслуживания

ρ = Mσ / Mτ = λ / μ — загрузка; система устойчива при ρ < 1 (здесь ρ = 4.5 / 5.5 ≈ 0.82)

M = Σw / n, D = $\Sigma(w - M)^2$ / n, σ = √D

Код

```cpp
#include <cmath>       // sqrt
#include <fstream>     // ofstream — запись в файл
#include <iomanip>
#include <iostream>
#include <random>
#include <vector>      // vector — динамический массив

// команда открытия Excel-файла зависит от ОС (выбирается при компиляции)
#if defined(_WIN32)
    #define OPEN_FILE_COMMAND "start \"\" waiting.xlsx"
#elif defined(__APPLE__)
    #define OPEN_FILE_COMMAND "open waiting.xlsx"
#else
    #define OPEN_FILE_COMMAND "xdg-open waiting.xlsx"
#endif

using namespace std;

const int maxTasks = 10000;        // сколько заданий моделируем

const double arrivalMin = 1.0;     // интервал между приходами tau ~ U(1, 10)
const double arrivalMax = 10.0;

const double serviceMin = 1.0;     // время обработки sigma ~ U(1, 8)
const double serviceMax = 8.0;

struct Task {
    int id;                        // номер задания
    double arrivalTime;            // когда пришло (чтобы потом посчитать ожидание)
};

int main() {
    random_device randomDevice;
    mt19937 randomGenerator(randomDevice());

    uniform_real_distribution<double> arrivalGenerator(
        arrivalMin, arrivalMax
    );

    uniform_real_distribution<double> serviceGenerator(
        serviceMin, serviceMax
    );

    vector<Task> queue;            // очередь (буфер) заданий, ждущих сервер
    vector<double> waitTimes;      // время ожидания каждого задания (0, если сразу в работу)

    int waiting[10] = {};          // гистограмма: waiting[i] — сколько ждали от i до i+1
    int waitingMoreThan10 = 0;     // сколько ждали 10 и больше

    int currentTask = 1;           // номер следующего приходящего задания
```

```
    int completedTasks = 0;        // сколько обработано
    int queuedTasks = 0;           // сколько попало в очередь
    size_t maxQueueSize = 0;       // максимальная длина очереди

    double totalWaitingTime = 0.0;

    double currentTime = 0.0;
    double incomingTime = arrivalGenerator(randomGenerator);   // момент ближайшего прихода
    double outgoingTime = 1e100;   // момент окончания обработки (1e100 = сервер свободен)

    bool busy = false;

    // крутимся, пока не пришли все задания и не опустели сервер и очередь
    while (currentTask <= maxTasks || busy || !queue.empty()) {

        if (currentTask <= maxTasks && incomingTime <= outgoingTime) {
            // ---------- событие IN: пришло задание ----------
            currentTime = incomingTime;

            Task task;
            task.id = currentTask;
            task.arrivalTime = currentTime;

            cout << fixed << setprecision(2);

            cout << "Пришла задача " << task.id
                 << " в момент " << currentTime << endl;

            incomingTime += arrivalGenerator(randomGenerator);   // планируем следующий приход
            currentTask++;

            if (!busy) {
                // сервер свободен — сразу в работу, ожидание 0
                busy = true;

                double serviceTime =
                    serviceGenerator(randomGenerator);

                outgoingTime = currentTime + serviceTime;        // когда закончит

                cout << "Задача " << task.id
                     << " сразу поступила в работу" << endl;

                waiting[0]++;
                waitTimes.push_back(0.0);
            }
            else {
                // сервер занят — в очередь
                queue.push_back(task);
                queuedTasks++;

                if (queue.size() > maxQueueSize) {
                    maxQueueSize = queue.size();
                }

                cout << "Задача " << task.id
                     << " добавлена в очередь" << endl;
            }
        }
        else {
            // ---------- событие OUT: сервер закончил обработку ----------
            currentTime = outgoingTime;

            completedTasks++;

            cout << fixed << setprecision(2);

            cout << "Обслуживание завершено в момент "
                 << currentTime << endl;

            busy = false;

            if (!queue.empty()) {

                // берём из очереди СЛУЧАЙНОЕ задание (а не первое)
                uniform_int_distribution<int> queueGenerator(
                    0,
                    static_cast<int>(queue.size()) - 1
                );

                int index = queueGenerator(randomGenerator);

```

```
                Task task = queue[index];

                queue.erase(queue.begin() + index);             // убираем его из очереди

                double waitingTime =
                    currentTime - task.arrivalTime;             // сколько простояло в очереди

                totalWaitingTime += waitingTime;
                waitTimes.push_back(waitingTime);

                // раскладываем по столбцам гистограммы шириной 1
                if (waitingTime < 1.0) {
                    waiting[0]++;
                }
                else if (waitingTime < 2.0) {
                    waiting[1]++;
                }
                else if (waitingTime < 3.0) {
                    waiting[2]++;
                }
                else if (waitingTime < 4.0) {
                    waiting[3]++;
                }
                else if (waitingTime < 5.0) {
                    waiting[4]++;
                }
                else if (waitingTime < 6.0) {
                    waiting[5]++;
                }
                else if (waitingTime < 7.0) {
                    waiting[6]++;
                }
                else if (waitingTime < 8.0) {
                    waiting[7]++;
                }
                else if (waitingTime < 9.0) {
                    waiting[8]++;
                }
                else if (waitingTime < 10.0) {
                    waiting[9]++;
                }
                else {
                    waitingMoreThan10++;
                }

                cout << "Задача " << task.id
                     << " взята из очереди" << endl;

                cout << "Время ожидания: "
                     << waitingTime << " сек." << endl;

                busy = true;                                    // сервер снова занят

                double serviceTime =
                    serviceGenerator(randomGenerator);

                outgoingTime =
                    currentTime + serviceTime;
            }
            else {
                outgoingTime = 1e100;                           // очередь пуста — сервер простаивает
            }
        }
    }

    // ---------- вывод результатов ----------
    cout << "\nРАСПРЕДЕЛЕНИЕ ВРЕМЕНИ ОЖИДАНИЯ\n";

    for (int i = 0; i < 10; i++) {
        cout << i << "–" << i + 1
             << " секунд: "
             << waiting[i]
             << " задач" << endl;
    }

    cout << "10+ секунд: "
         << waitingMoreThan10
         << " задач" << endl;

    cout << "\nСТАТИСТИКА\n";

    cout << "Всего задач: "
```

```
215          << maxTasks << endl;
216
217      cout << "Задач попало в очередь: "
218           << queuedTasks << endl;
219
220      cout << "Завершено задач: "
221           << completedTasks << endl;
222
223      if (queuedTasks > 0) {
224          cout << fixed << setprecision(2);
225
226          cout << "Среднее время ожидания (среди задач из очереди): "
227               << totalWaitingTime / queuedTasks
228               << " секунд" << endl;
229      }
230
231      // параметры системы
232      double meanTau = (arrivalMin + arrivalMax) / 2.0;   // Mtau — средний интервал прихода
233      double meanSigma = (serviceMin + serviceMax) / 2.0; // Msigma — среднее время обработки
234      double rho = meanSigma / meanTau;                   // загрузка; система устойчива при rho < 1
235
236      // мат. ожидание, дисперсия и СКО времени ожидания по ВСЕМ задачам
237      double waitSum = 0.0, maxWait = 0.0;
238      int zeroWaitCount = 0;
239      for (double w : waitTimes) {
240          waitSum += w;
241          if (w > maxWait) maxWait = w;
242          if (w == 0.0) zeroWaitCount++;                  // не ждали вообще
243      }
244      double waitMean = waitSum / waitTimes.size();       // M = сумма / количество
245
246      double waitSq = 0.0;
247      for (double w : waitTimes) {
248          waitSq += (w - waitMean) * (w - waitMean);
249      }
250      double waitDisp = waitSq / waitTimes.size();        // D = среднее (w - M)^2
251
252      cout << fixed << setprecision(3);
253
254      cout << "\nПараметры\n";
255      cout << "загрузка rho:           " << rho << endl;
256      cout << "lambda (интенс. прихода): " << 1.0 / meanTau << endl;     // lambda = 1 / Mtau
257      cout << "mu (интенс. обслуж.):     " << 1.0 / meanSigma << endl;   // mu = 1 / Msigma
258
259      cout << "\nСтатистика по времени ожидания (все задачи)\n";
260      cout << "мат. ожидание:          " << waitMean << endl;
261      cout << "дисперсия:              " << waitDisp << endl;
262      cout << "ср.кв. отклонение:      " << sqrt(waitDisp) << endl;      // sigma = sqrt(D)
263      cout << "максимальное ожидание:  " << maxWait << endl;
264      cout << "доля без ожидания:      "
265           << (double)zeroWaitCount / waitTimes.size() << endl;
266      cout << "макс. размер очереди:   " << maxQueueSize << endl;
267
268      // ---------- Excel: пишем данные в файл и генерируем python-скрипт ----------
269      ofstream dataFile("waiting_data.txt");
270
271      for (int i = 0; i < 10; i++) {
272          dataFile << waiting[i] << " ";              // 11 чисел: столбцы гистограммы
273      }
274
275      dataFile << waitingMoreThan10;
276
277      dataFile.close();
278
279      ofstream pythonFile("create_excel.py");
280
281      // скрипт читает гистограмму, считает накопленную долю F(x) и строит график
282      pythonFile << "from openpyxl import Workbook\n";
283      pythonFile << "from openpyxl.chart import LineChart, Reference\n";
284      pythonFile << "\n";
285
286      pythonFile << "with open('waiting_data.txt', 'r') as file:\n";
287      pythonFile << "    values = list(map(int, file.read().split()))\n";
288      pythonFile << "\n";
289
290      pythonFile << "intervals = [\n";
291      pythonFile << "    '0-1', '1-2', '2-3', '3-4', '4-5',\n";
292      pythonFile << "    '5-6', '6-7', '7-8', '8-9', '9-10', '10+'\n";
293      pythonFile << "]\n";
294      pythonFile << "\n";
295
296      pythonFile << "total = sum(values)\n";
```

```
    pythonFile << "\n";

    pythonFile << "workbook = Workbook()\n";
    pythonFile << "sheet = workbook.active\n";
    pythonFile << "sheet.title = 'Функция распределения'\n";
    pythonFile << "\n";

    pythonFile << "sheet['A1'] = 'Время ожидания'\n";
    pythonFile << "sheet['B1'] = 'F(x)'\n";
    pythonFile << "\n";

    pythonFile << "cumulative = 0\n";
    pythonFile << "\n";

    // F(x) = доля заданий, которые ждали не дольше x (накопленная сумма / всего)
    pythonFile << "for i in range(11):\n";
    pythonFile << "    cumulative += values[i]\n";
    pythonFile << "    sheet.cell(row=i + 2, column=1, value=intervals[i])\n";
    pythonFile << "    sheet.cell(row=i + 2, column=2, value=cumulative / total)\n";
    pythonFile << "\n";

    pythonFile << "chart = LineChart()\n";
    pythonFile << "chart.title = 'Функция распределения времени ожидания'\n";
    pythonFile << "chart.x_axis.title = 'Время ожидания'\n";
    pythonFile << "chart.y_axis.title = 'F(x)'\n";
    pythonFile << "\n";

    pythonFile << "data = Reference(sheet, min_col=2, min_row=1, max_row=12)\n";
    pythonFile << "categories = Reference(sheet, min_col=1, min_row=2, max_row=12)\n";
    pythonFile << "\n";

    pythonFile << "chart.add_data(data, titles_from_data=True)\n";
    pythonFile << "chart.set_categories(categories)\n";
    pythonFile << "\n";

    pythonFile << "chart.width = 20\n";
    pythonFile << "chart.height = 10\n";
    pythonFile << "\n";

    pythonFile << "sheet.add_chart(chart, 'D2')\n";
    pythonFile << "\n";

    pythonFile << "workbook.save('waiting.xlsx')\n";

    pythonFile.close();

    system("python3 create_excel.py");     // запускаем скрипт — он создаёт waiting.xlsx

    cout << "\nExcel-файл waiting.xlsx создан" << endl;

    system(OPEN_FILE_COMMAND);             // открываем Excel-файл

    return 0;
}
```

Лаб. 4 — Суперпозиция потоков

`lab4.cpp`

Задача

N независимых потоков, интервалы в каждом ~ U(a, b). Все события сливаются в один итоговый поток. Найти интенсивность итогового потока, средний интервал между соседними событиями и плотность распределения интервалов; сравнить с пуассоновским потоком.

Как работает

- Для каждого потока храним момент его следующего события **nextEvent[i]**.
- Очередное событие итогового потока — самое раннее из них. Записываем интервал от предыдущего события и разыгрываем этому потоку новое событие.
- По лекции 3: сумма многих непуассоновских потоков близка к пуассоновскому. Чем больше N, тем ближе плотность к $\lambda e^{-\lambda x}$ и коэффициент вариации к 1.
- Интервалы итогового потока бывают от 0 (события разных потоков рядом) до b.

Формулы

$\lambda_1 = 1 / M\tau = 2 / (a + b)$, $\lambda_{сум} = N \cdot \lambda_1$, средний интервал $= 1 / \lambda_{сум}$

$D = M[x^2] - (M[x])^2$, $\sigma = \sqrt{D}$, коэф. вариации $= \sigma / M$ (у пуассоновского = 1)

$f(x) \approx$ (число интервалов в столбце) / (всего · ширина столбца)

пуассоновский поток: $f(x) = \lambda e^{-\lambda x}$, $P\{x \text{ в } [c; d)\} = e^{-\lambda c} - e^{-\lambda d}$

Код

```cpp
#include <cmath>       // sqrt, exp
#include <fstream>
#include <iomanip>
#include <iostream>
#include <random>
#include <vector>

// команды запуска python и открытия файла зависят от ОС
#if defined(_WIN32)
    #define PYTHON_COMMAND "python create_excel_lab4.py"
    #define OPEN_FILE_COMMAND "start \"\" flows.xlsx"
#elif defined(__APPLE__)
    #define PYTHON_COMMAND "python3 create_excel_lab4.py"
    #define OPEN_FILE_COMMAND "open flows.xlsx"
#else
    #define PYTHON_COMMAND "python3 create_excel_lab4.py"
    #define OPEN_FILE_COMMAND "xdg-open flows.xlsx"
#endif

using namespace std;

const int N = 5;                  // число потоков
const double a = 0.0;             // интервалы в каждом потоке ~ U(a, b)
const double b = 8.0;

const int totalEvents = 100000;   // сколько событий суммарного потока моделируем
const int binCount = 20;          // число столбцов гистограммы

int main() {
    random_device randomDevice;
    mt19937 randomGenerator(randomDevice());

    uniform_real_distribution<double> intervalGenerator(a, b);

    // nextEvent[i] — момент следующего события в потоке i
    vector<double> nextEvent(N);
    for (int i = 0; i < N; i++) {
        nextEvent[i] = intervalGenerator(randomGenerator);
    }

    double binWidth = b / binCount;   // ширина столбца (интервал суммы не бывает больше b)
    vector<int> hist(binCount, 0);    // hist[k] — сколько интервалов попало в столбец k

    double sum = 0.0;                 // сумма интервалов — для среднего
    double sumSq = 0.0;               // сумма КВАДРАТОВ интервалов — для дисперсии
    double lastEventTime = 0.0;       // момент предыдущего события суммарного потока

```

```
    for (int event = 0; event < totalEvents; event++) {
        // очередное событие суммарного потока = самое раннее среди всех потоков
        int flow = 0;
        for (int i = 1; i < N; i++) {
            if (nextEvent[i] < nextEvent[flow]) {
                flow = i;
            }
        }

        double currentTime = nextEvent[flow];
        double interval = currentTime - lastEventTime;   // интервал между соседними событиями

        sum += interval;
        sumSq += interval * interval;

        int k = (int)(interval / binWidth);              // в какой столбец попал интервал
        if (k >= binCount) k = binCount - 1;             // защита от выхода за массив
        hist[k]++;

        lastEventTime = currentTime;
        // у этого потока событие произошло — разыгрываем его следующее событие
        nextEvent[flow] = currentTime + intervalGenerator(randomGenerator);
    }

    // теория: интенсивность суммы потоков = сумма интенсивностей
    double lambdaOne = 2.0 / (a + b);       // lambda = 1 / Mtau = 1 / ((a + b) / 2)
    double lambdaSum = N * lambdaOne;       // lambda_сум = N * lambda

    // практика
    double mean = sum / totalEvents;                    // M — средний интервал
    double disp = sumSq / totalEvents - mean * mean;    // D = M[x^2] - (M[x])^2
    double sigma = sqrt(disp);                          // СКО

    cout << fixed << setprecision(4);

    cout << "N = " << N << ", a = " << a << ", b = " << b << endl;

    cout << "\nДолжно быть в теории\n";
    cout << "интенсивность одного потока:  " << lambdaOne << endl;
    cout << "интенсивность суммарного:     " << lambdaSum << endl;
    cout << "средний интервал суммарного:  " << 1.0 / lambdaSum << endl;

    cout << "\nНа практике\n";
    cout << "интенсивность суммарного:     " << totalEvents / lastEventTime << endl;  // событий / время
    cout << "средний интервал:             " << mean << endl;
    cout << "дисперсия:                    " << disp << endl;
    cout << "ср.кв. отклонение:            " << sigma << endl;
    // у пуассоновского потока sigma = M, поэтому sigma / M = 1
    cout << "коэф. вариации sigma/M:       " << sigma / mean
         << "  (у пуассоновского = 1)" << endl;

    cout << "\nПлотность распределения\n";
    cout << "   интервал        f(x) модел.   % интерв.   f(x) пуассон   % интерв.\n";

    ofstream dataFile("flows_data.txt");

    for (int k = 0; k < binCount; k++) {
        double from = k * binWidth;                     // левая граница столбца
        double to = (k + 1) * binWidth;                 // правая граница
        double x = (k + 0.5) * binWidth;                // середина — для графика

        // плотность = доля попавших / ширина столбца
        double density = hist[k] / (totalEvents * binWidth);
        double percent = 100.0 * hist[k] / totalEvents;

        // плотность пуассоновского потока: f(x) = lambda * e^(-lambda * x)
        double poissonDensity = lambdaSum * exp(-lambdaSum * x);
        // вероятность попасть в [from; to) у пуассоновского потока: F(to) - F(from)
        double poissonPercent = 100.0 * (exp(-lambdaSum * from) - exp(-lambdaSum * to));

        cout << setprecision(1)
             << "[" << setw(4) << from << "; " << setw(4) << to << ")   "
             << setprecision(4) << setw(10) << density << "   "
             << setprecision(2) << setw(7) << percent << " %   "
             << setprecision(4) << setw(10) << poissonDensity << "   "
             << setprecision(2) << setw(7) << poissonPercent << " %" << endl;

        dataFile << x << " " << density << " " << poissonDensity << "\n";
    }

    dataFile.close();
```

```
    // python-скрипт: читает flows_data.txt и строит график двух плотностей
    ofstream pythonFile("create_excel_lab4.py");

    pythonFile << "from openpyxl import Workbook\n";
    pythonFile << "from openpyxl.chart import LineChart, Reference\n";
    pythonFile << "\n";
    pythonFile << "workbook = Workbook()\n";
    pythonFile << "sheet = workbook.active\n";
    pythonFile << "sheet.append(['x', 'f(x) моделирование', 'f(x) пуассоновский'])\n";
    pythonFile << "\n";
    pythonFile << "with open('flows_data.txt') as file:\n";
    pythonFile << "    for line in file:\n";
    pythonFile << "        sheet.append([round(float(v), 4) for v in line.split()])\n";
    pythonFile << "\n";
    pythonFile << "chart = LineChart()\n";
    pythonFile << "chart.title = 'Плотность распределения интервалов суммарного потока'\n";
    pythonFile << "chart.x_axis.title = 'x'\n";
    pythonFile << "chart.y_axis.title = 'f(x)'\n";
    pythonFile << "chart.add_data(Reference(sheet, min_col=2, max_col=3, min_row=1,"
                  " max_row=sheet.max_row), titles_from_data=True)\n";
    pythonFile << "chart.set_categories(Reference(sheet, min_col=1, min_row=2,"
                  " max_row=sheet.max_row))\n";
    pythonFile << "chart.width = 20\n";
    pythonFile << "chart.height = 10\n";
    pythonFile << "sheet.add_chart(chart, 'E2')\n";
    pythonFile << "\n";
    pythonFile << "workbook.save('flows.xlsx')\n";

    pythonFile.close();

    system(PYTHON_COMMAND);                // создаём flows.xlsx

    cout << "\nExcel-файл flows.xlsx создан" << endl;

    system(OPEN_FILE_COMMAND);             // и открываем его

    return 0;
}
```

Лаб. 5 — Двухфазная система

lab5.cpp

Задача

Два сервера, у каждого своя очередь FIFO. Обработка фиксированная: σ1 = 11 на 1 фазе, σ2 = 17 на 2 фазе. Каждому заданию случайно выдаются k из {1, 2, 3} и m из {1, 2}: задание k раз проходит сервер 1 (каждый раз возвращаясь в конец очереди 1), затем m раз сервер 2 и уходит. Найти условие устойчивости (какими могут быть a и b) и среднее время пребывания в системе.

Как работает

- Три события: приход, конец цикла на сервере 1, конец цикла на сервере 2 — берём самое раннее.
- Очередь 2 нужна, чтобы задание дождалось сервер 2, если он занят.
- Время пребывания = момент ухода после последнего цикла 2 фазы − момент первого прихода.
- Загрузка из моделирования = время работы сервера / общее время.

Формулы

M[k] = (1 + 3) / 2 = 2, M[m] = (1 + 2) / 2 = 1.5 (значения равновероятны)

ρ_1 = M[k]·σ1 / Mт = 22 / Mт, ρ_2 = M[m]·σ2 / Mт = 25.5 / Mт

устойчивость: $\rho_1 < 1$ и $\rho_2 < 1$ → Mт = (a + b) / 2 > 25.5 → **a + b > 51**

без очередей время пребывания было бы 22 + 25.5 = 47.5; при a = 20, b = 40 выходит ≈ 65

Код

```cpp
#include <deque>       // deque — очередь (быстро снимать с начала и класть в конец)
#include <iomanip>
#include <iostream>
#include <random>

using namespace std;

const double sigma1 = 11.0;       // время обработки на 1 фазе (фиксированное)
const double sigma2 = 17.0;       // время обработки на 2 фазе

const int maxK = 3;               // k от 1 до 3 — сколько циклов на 1 фазе
const int maxM = 2;               // m от 1 до 2 — сколько циклов на 2 фазе

const int totalTasks = 10000;     // сколько заданий моделируем

struct Task {
    double arrivalTime;           // момент прихода в систему (для времени пребывания)
    int k;                        // сколько циклов осталось на 1 фазе
    int m;                        // сколько циклов осталось на 2 фазе
};

int main() {
    double a, b;                  // интервалы между приходами заданий ~ U(a, b)
    cout << "Введите a и b: ";
    cin >> a >> b;

    if (a < 0 || b <= a) {
        cout << "Нужно 0 <= a < b" << endl;
        return 1;
    }

    // ---------- теория: условие устойчивости ----------
    double meanK = (1 + maxK) / 2.0;     // M[k] = (1 + 3) / 2 = 2 (k равновероятно 1..3)
    double meanM = (1 + maxM) / 2.0;     // M[m] = (1 + 2) / 2 = 1.5; 2.0 — чтобы деление было дробным
    double work1 = meanK * sigma1;       // работа сервера 1 на одно задание: 2 * 11 = 22
    double work2 = meanM * sigma2;       // работа сервера 2 на одно задание: 1.5 * 17 = 25.5
    double meanTau = (a + b) / 2.0;      // Mtau — средний интервал между приходами

    // rho = work / Mtau < 1 у обоих серверов => Mtau больше большей из работ
    double minTau = work1;
    if (work2 > minTau) {
        minTau = work2;
    }

    cout << "Условие устойчивости: a + b > " << 2 * minTau << endl;
    cout << "  сервер 1: M[k] * sigma1 = " << meanK << " * " << sigma1 << " = " << work1 << endl;
    cout << "  сервер 2: M[m] * sigma2 = " << meanM << " * " << sigma2 << " = " << work2 << endl;
    cout << "  берём большее: (a + b) / 2 > " << minTau << ", значит a + b > " << 2 * minTau << endl;
```

```
49
50      cout << fixed << setprecision(3);
51
52      cout << "\na = " << a << ", b = " << b << ", Mtau = " << meanTau << endl;
53      cout << "rho1 = " << work1 / meanTau << ", rho2 = " << work2 / meanTau << endl;
54
55      if (meanTau > minTau) {
56          cout << "система устойчива" << endl;
57      }
58      else {
59          cout << "система неустойчива — очередь будет расти бесконечно" << endl;
60      }
61
62      // ---------- моделирование ----------
63      random_device randomDevice;
64      mt19937 randomGenerator(randomDevice());
65
66      uniform_real_distribution<double> arrivalGenerator(a, b);
67      uniform_int_distribution<int> kGenerator(1, maxK);       // k равновероятно 1, 2, 3
68      uniform_int_distribution<int> mGenerator(1, maxM);       // m равновероятно 1, 2
69
70      deque<Task> queue1;           // очередь к серверу 1 (FIFO)
71      deque<Task> queue2;           // очередь к серверу 2 — ждут, пока освободится сервер 2
72
73      Task task1;                   // задание на сервере 1
74      Task task2;                   // задание на сервере 2
75      bool busy1 = false;
76      bool busy2 = false;
77
78      // три вида событий: приход, конец обработки на сервере 1, на сервере 2
79      double incomingTime = arrivalGenerator(randomGenerator);
80      double outgoingTime1 = 1e100; // 1e100 — сервер свободен (событие «никогда»)
81      double outgoingTime2 = 1e100;
82
83      double currentTime = 0.0;
84      double busyTime1 = 0.0;       // сколько времени сервер 1 работал — для загрузки
85      double busyTime2 = 0.0;
86      double sumTime = 0.0;         // сумма времён пребывания — для среднего
87
88      int arrived = 0;              // сколько заданий пришло
89      int finished = 0;             // сколько ушло из системы
90
91      while (finished < totalTasks) {
92
93          // обрабатываем самое раннее из трёх событий
94          if (incomingTime <= outgoingTime1 && incomingTime <= outgoingTime2) {
95              // пришло новое задание — в очередь 1 фазы
96              currentTime = incomingTime;
97              arrived++;
98
99              Task task;
100             task.arrivalTime = currentTime;
101             task.k = kGenerator(randomGenerator);
102             task.m = mGenerator(randomGenerator);
103             queue1.push_back(task);
104
105             if (arrived < totalTasks) {
106                 incomingTime = currentTime + arrivalGenerator(randomGenerator);
107             }
108             else {
109                 incomingTime = 1e100;              // все задания уже пришли
110             }
111         }
112         else if (outgoingTime1 <= outgoingTime2) {
113             // сервер 1 закончил цикл
114             currentTime = outgoingTime1;
115             busy1 = false;
116             outgoingTime1 = 1e100;
117
118             task1.k--;                             // один цикл 1 фазы пройден
119             if (task1.k > 0) {
120                 queue1.push_back(task1);           // ещё раз на 1 фазу, в конец очереди
121             }
122             else {
123                 queue2.push_back(task1);           // 1 фаза закончена — на 2 фазу
124             }
125         }
126         else {
127             // сервер 2 закончил цикл
128             currentTime = outgoingTime2;
129             busy2 = false;
130             outgoingTime2 = 1e100;
```

```
131
132             task2.m--;                              // один цикл 2 фазы пройден
133             if (task2.m > 0) {
134                 queue2.push_back(task2);            // ещё раз на 2 фазу
135             }
136             else {
137                 // задание уходит из системы: время пребывания = сейчас − момент прихода
138                 sumTime += currentTime - task2.arrivalTime;
139                 finished++;
140             }
141         }
142
143         // свободный сервер берёт первое задание из своей очереди
144         if (!busy1 && !queue1.empty()) {
145             task1 = queue1.front();
146             queue1.pop_front();
147             busy1 = true;
148             outgoingTime1 = currentTime + sigma1;  // закончит через sigma1
149             busyTime1 += sigma1;
150         }
151
152         if (!busy2 && !queue2.empty()) {
153             task2 = queue2.front();
154             queue2.pop_front();
155             busy2 = true;
156             outgoingTime2 = currentTime + sigma2;
157             busyTime2 += sigma2;
158         }
159     }
160
161     cout << "\nМоделирование\n";
162     cout << "загрузка сервера 1:        " << busyTime1 / currentTime << endl;   // доля времени в работе
163     cout << "загрузка сервера 2:        " << busyTime2 / currentTime << endl;
164     cout << "среднее время пребывания:  " << sumTime / totalTasks << endl;
165     cout << "только обработка:          " << work1 + work2 << endl;           // 22 + 25.5 = 47.5
166
167     return 0;
168 }
```